```
NUM _ THREADS”] = “NUM _ PARALLEL _ EXEC _ UNITS”
os.environ[“KMP _ BLOCKTIME”] = “30”
•          os.environ[“KMP _ SETTINGS”] = “1”
os.environ[“KMP _ AFFINITY”]= “granularity=fine,verbose,com
pact,1,0”
```

7. Running the Keras functional API on a sample of the entire dataset before applying the model to the entire data is a good method to avoid crashes and hang ups.
8. Keep secondary cross validation procedures vacant while the main processes run to reduce pressure on the hardware. Maintain a hardware capacity, whether a CPU or GPU, below 85%. Larger PCs can cross 90% but for beginners use smaller predetermined limits.

Installing CUDA drivers

```
• C:\cudnn-9.0-windows10-x64-v7,C:\cudnn-9.0-windows10-x64-v7\
cuda\bin
• conda create -n tensorflow pip python=3.6, activate ten-
sorflow
• pip install tensorflow-gpu==1.8.0, sudo cp lib64/* /usr/
local/cuda/lib64/
• sudo cp include/cudnn.h /usr/local/cuda/include/, export
LD _ LIBRARY _ PATH=/usr/local/cuda/lib64:${LD _ LIBRARY _ PATH}
• export PATH=/usr/local/cuda/lib64:${PATH}
```

The CUDA drivers can also be used to perform Kernel optimization using an iterative loop. This is done by first creating an inner loop that has a scalable RPC runtime framework and a tensor compiler. The tuner picks a batch of kernel implementations that have promising candidates and then impose them on a real hardware. The turner then extracts the profiling results and are later used as training data to fit the prediction model. After fitting the prediction model, the tuner proceeds to pick the next best candidates according to the predictions, and the loop continues. Searching for the best kernels thus becomes faster.

```
• cudaEvent _ t start, stop; float time;
cudaEventCreate(&start); cudaEventCreate(&stop);
• cudaEventRecord( start, 0 ); kernel<<<grid,threads>>> (
d _ odata, d _ idata, size _ x, size _ y,
•  NUM _ REPS); cudaEventRecord( stop, 0 ); cudaEventSyn-
chronize( stop ); cudaEventElapsedTime( &time, start, stop
```